GEbPDCM + LTPM

Viva Script

A read-aloud walkthrough of the project, timed for roughly 6-8 minutes, followed by a one-page quick-answer sheet for the Q&A that follows.

How to use this: read the plain paragraphs as your spoken script. The italic lines in brackets are stage directions -- point at a file, a graph, or a table on screen at that moment. Practice it out loud 2-3 times before the viva; you do not need to memorise it word for word, just know the shape of each section.

1. Opening (30 seconds)

Good [morning/afternoon]. My project reproduces and extends GEbPDCM -- Golden Eagle-based Primal-Dual Congestion Management -- a congestion-control protocol from a 2025 paper by Sandhya and Joshi, published in Multimedia Tools and Applications. The protocol targets mobile ad-hoc networks running AODV routing, where some nodes end up relaying more traffic than others and become congestion hotspots.

I built two things on top of the paper's base scheme: a real Golden Eagle Optimizer, and a Lightweight Trust and Predictive Module -- both of which the paper names in its title but never actually implements. I then tested everything in two independent simulations, and found something the paper's own framing doesn't anticipate. That's what I'll walk through.

2. The Problem the Paper Addresses (1 minute)

In a mobile ad-hoc network, every node can act as both a sender and a relay. As traffic and mobility increase, some nodes end up sitting on more forwarding paths than others -- either because of the topology, or just temporary clustering. Those nodes' queues fill up, delay spikes, packets get dropped, and retransmissions burn energy without improving delivery. Plain AODV has no mechanism to notice or correct this -- it only finds routes, it doesn't relieve pressure once one exists.

GEbPDCM's fix, in plain terms: continuously estimate each node's load and how crowded its neighbourhood is, combine those into a congestion score, and once a node crosses a threshold, move half its load to a nearby node that isn't itself congested. That's the entire algorithm the paper actually specifies.

3. The Gap in the Paper (1 minute)

[This is the finding to lead with if the examiner asks 'what did you actually contribute?']

The paper's title and abstract promise two extra techniques: a Golden Eagle Optimizer for choosing which node to migrate to, and a primal-dual mathematical formulation of the whole problem. I read the paper's method section carefully, and neither one is actually there. There's no population of candidate solutions, no fitness landscape, no attack-and-cruise search dynamics -- the 'optimizer' is just a threshold check and 'pick the nearest free node.' There's no Lagrangian, no dual variables anywhere either.

I also found two of the paper's own equations are dimensionally broken -- Equation 2 adds a set of nodes to a data rate, and Equation 3 adds a data rate to a neighbour-count term. Neither is

addable as literally written. I documented my own interpretation of both in the code, rather than silently fixing them and pretending that's what the paper meant.

So the real GEO implementation and the LTPM module aren't just extra features -- they're literally what the paper's own title claims to contain, built for real.

4. What I Built (2 minutes)

4.1 GEO -- Golden Eagle Optimizer

[If demoing live: open geo.py or gebpdcm.cc's GeoSelect function here]

Each candidate migration target is treated like a point an eagle can fly towards. Every iteration, each of 20 'eagles' moves under two combined vectors: an attack vector pointing straight at the best candidate found so far -- that's exploitation -- and a cruise vector perpendicular to it, circling for something better -- that's exploration. Early on, cruising dominates; by the last of 30 iterations, attack dominates, matching how a real eagle commits to a strike as a hunt progresses. Each candidate is scored by a fitness function combining headroom, delivery quality, distance, current load, and remaining energy.

4.2 LTPM -- Lightweight Trust and Predictive Module

This is my own addition, not something the paper names at all. It has two parts. First, trust scoring: every node has an EWMA trust score that rises when it forwards cleanly and falls when it doesn't; a node is only eligible as a migration target once its trust clears a threshold -- this stops the protocol from routing more load onto an unreliable or malicious node. Second, predictive forecasting: a simple linear extrapolation over each node's recent load history lets the protocol migrate load before a node congests, not just after. Deliberately no machine learning here -- the module is called Lightweight, and a trained model isn't defensible when simple arithmetic solves the actual problem.

4.3 Two independent implementations

I built this twice. Version 2 is a pure Python model -- I wrote this first, to work out and validate every equation quickly. Version 1 ports the identical logic into a real NS-3 packet-level simulation -- real WiFi queues, real AODV routing, real packets. Both run on the exact same 100-node topology and the same four schemes: baseline, GEbPDCM as written, GEbPDCM plus GEO, and GEbPDCM plus GEO plus LTPM.

5. Results (2 minutes)

[Open the comparison tables / graphs here -- results/comparison_table.csv in each folder]

In Version 2, the Python model, the ordering is exactly what you'd expect from the paper's own narrative: GEbPDCM plus GEO plus LTPM has the lowest delay, highest throughput, and highest packet delivery ratio of all four schemes, at every mobility speed I tested, from 10 to 50 metres per second.

Method	Delay (ms)	Throughput (Mbps)	PDR
Baseline (AODV)	49.94	78.54	0.9067
GEbPDCM	49.92	78.54	0.9068
GEbPDCM + GEO	49.91	78.55	0.9068
GEbPDCM + GEO + LTPM (ours)	49.58	78.64	0.9078

[point at v2's table] Here, LTPM wins on every metric, as expected.

Version 1, the real NS-3 simulation, tells a completely different story. Plain baseline -- no congestion control at all -- beats all three congestion-controlled schemes, at every single speed.

Method	Delay (ms)	Throughput (Mbps)	PDR
Baseline (AODV)	27.58	0.31	0.2626
GEbPDCM	52.62	0.22	0.1852
GEbPDCM + GEO	51.28	0.21	0.1737
GEbPDCM + GEO + LTPM (ours)	48.07	0.21	0.1736

[point at v1's table] Baseline has the lowest delay and the highest throughput and PDR. GEO and LTPM do still improve on plain GEbPDCM -- you can see LTPM has the lowest delay of the three congestion-controlled schemes -- but none of them close the gap back to simply not migrating at all.

6. Why the Two Versions Disagree -- the Core Finding (2 minutes)

[This is the single most important thing to get across. Slow down here.]

Both versions run the identical topology, the identical equations, the identical thresholds. The difference is not a bug in either one -- it comes from a structural assumption the two models make differently about what 'network capacity' actually is.

In the Python model, every node's load is an independent number, scored against a fixed threshold independently of every other node. When I migrate load from an overloaded node to an underloaded one, I directly lower the sender's score and raise the receiver's, with no cost anywhere else. Moving load between independent buckets always helps, by construction.

In the real NS-3 simulation, all 100 nodes share one physical 2 megabit-per-second wireless channel. There is no such thing as 'node B's spare capacity' independent of node A's -- every transmission from any node in range consumes the same shared airtime. When I reroute a congested node's traffic through a neighbour, that packet is now transmitted twice instead of once -- that's strictly more load on the same contended channel, not less. On top of that, the

congestion score is heavily driven by how many neighbours are in range, which migration can't change at all, since that's a property of the topology, not of any one node's traffic.

So the Python model matches the setting the paper's own equations implicitly assume -- separable capacity, like independent wired links. The real NS-3 test exposes that this assumption doesn't hold on an actual shared wireless channel. That's a genuine finding about the paper's underlying design, and I've reported it exactly as measured -- I did not retune anything to make GEO and LTPM win in NS-3 too, because that would have hidden the most important result of the whole project.

7. Conclusion (30 seconds)

To summarise: I reproduced the paper's GEbPDCM scheme, built the Golden Eagle Optimizer and the primal-dual framing the paper claims but never implements -- well, GEO for real, and I found there genuinely is no primal-dual formulation to build, which I documented as a finding rather than invented one to satisfy the title. I added LTPM as a genuine novelty for security and predictive migration. And by testing the same algorithm in two independently-built simulations, I found and explained a real limitation in the paper's own model: on a real shared wireless channel, migration doesn't relieve congestion the way an abstracted model suggests it should. That's the project.

Quick-Answer Sheet (glance at this right before you go in)

If asked...	Say...
What's your novelty?	Real GEO (attack/cruise search) + LTPM (trust EWMA + linear forecast) -- both named by the paper's title, neither built by the paper's method section.
Why does GEO/LTPM lose to baseline in NS-3?	Real wireless capacity is one shared channel, not independent buckets -- rerouting adds a hop on the SAME contended medium, it doesn't add capacity. Topology-driven congestion (neighbour density) can't be fixed by migration at all.
Then why keep GEO/LTPM in the project?	They still beat plain GEbPDCM in both versions -- a real improvement over the paper's own design under identical constraints. The limit is in migration-as-a-strategy on a shared channel, not in smarter target selection.
Why two versions?	Python validates the equations fast, independent of simulator quirks. NS-3 tests the algorithm against a real, physically grounded network. Building both and finding they disagree was the most valuable outcome.
What's the primal-dual part?	There isn't one in the paper -- no Lagrangian, no dual variables, no update rule anywhere in the method section. Documented as a finding, not invented to match the title.
Are your results satisfying?	Partially, and I say so directly: neither version reproduces the paper's absolute numbers exactly (expected, since the paper's own simulation code isn't published), but v2 validates the equations behave as intended, and v1 surfaces a genuine, honestly-reported limitation in the paper's design.
Why NS-3 initially gave 0% delivery?	A real bug in this ns-3.42 build: HR/DSSS WiFi frames were logged as noise and silently discarded at receive time. Found via a debug-profile rebuild with NS_LOG enabled; fixed by switching to plain DSSS (DsssRate2Mbps).
What would you do with more time?	Test on a scenario with genuinely separable capacity -- multiple channels, or a wired backhaul per cluster -- which is the setting the paper's model assumes, and where GEO/LTPM would be expected to close the NS-3 gap too.